Assignment 8: Implementing STLC

CAS CS 320: Concepts of Programming Languages

due Thursday, November 5 by 8:00PM

For the OCaml problems, you'll put your solutions in the file `lib/assign8.ml`. See the file `test/test_assign8.ml` for example behavior of each function. You'll submit your solutions via Gradescope, where your code will be checked against a collection of hidden tests. You must pass **3/4th** of the Gradescope unit tests on each problem for the assignment to be considered attempted. Your code must also follow the guidelines of the course for it to be considered attempted (i.e., it may not use any references, arrays, mutable fields, or for/while loops).

For the written problems, you'll submit your solutions via Gradescope as a single PDF file. They must be hand-written and exceptionally neat, and the final answer to each problem must be clear, e.g., surrounded by a box. You must cite your sources per the instructions in the course manual.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problems with as few external aids as possible.* You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

Programming

Outline. This week you'll be implementing the simply-typed λ-calculus (STLC) with let-expressions and natural numbers via substitution semantics. You'll also be "programming" in STLC. This is what you need to implement.

```
module Expr : sig
  type t
  val of_sexpr_opt : Sexpr.t -> t option
  val subst : t -> string -> t -> t
end
val type_of : Expr.t -> (Type.t, string) result (* bolded *)
val eval : Expr.t -> Expr.t (* bolded *)
val multipy : Expr.t
```

Syntax

We'll again be using S-expressions for the concrete syntax. Note the introduction of types into the syntax.

```
  <ty> ::= Nat | ( -> <ty> <ty> )
<expr> ::= ( lambda <var> <ty> <expr> )
         | ( <expr> <expr> )
         | ( let <var> <expr> <expr> )
         | ( add1 <expr> )
         | ( rec-Nat <expr> <expr> <expr> )
         | <nat>
         | <var>
```

where `<nat>` is a nonempty sequence of digits and `<var>` is a nonempty sequence of lowercase English letters or underscores. You need to implement:

```
val of_sexpr_opt : Sexpr.t -> Expr.t option
```

so that `of_sexpr_opt se` is `Some e` if `se` is recognized by the above grammar and represents the expression e, and is `None` otherwise. A couple notes:

- The concrete syntax `add1` corresponds to the constructor `Succ`.
- Natural numbers (`<nat>`) represents stacks of `Succ` constructors on top of `Zero`, e.g.,

```
let _ = assert (Expr.of_sexpr_opt (Atom "4")
                = Some (Succ (Succ (Succ (Succ Zero)))))
```

Typing

The typing rules are given in terms of the following abstract syntax:

$$\begin{array}{rcl} \tau & ::= & \mathbb{N} \mid \tau \to \tau \\ e & ::= & \lambda x^{\tau}.e \mid ee \mid \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \\ & \mid & \mathsf{Z} \mid \mathsf{S}(e) \mid \mathsf{rec}\mathbb{N}(e, e, e) \\ & \mid & x \end{array}$$

Typing rules:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x^{\tau_1}.e : \tau_1 \to \tau_2}\ \textsc{fun}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \to \tau \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau}\ \textsc{app} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2}\ \textsc{let}$$

$$\frac{}{\Gamma \vdash \mathsf{Z} : \mathbb{N}}\ \textsc{zero} \qquad \frac{\Gamma \vdash e : \mathbb{N}}{\Gamma \vdash \mathsf{S}(e) : \mathbb{N}}\ \textsc{succ}$$

$$\frac{\Gamma \vdash e_1 : \mathbb{N} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \mathbb{N} \to \tau \to \tau}{\Gamma \vdash \mathsf{rec}\mathbb{N}(e_1, e_2, e_3) : \tau}\ \textsc{recNat}$$

You need to implement:

```
val type_of : Ctxt.t -> Expr.t -> (Type.t, string) result
```

so that `type_of gamma e` is `Ok t` if e has type t in the context Γ, i.e., $\Gamma \vdash e : t$ is derivable according to the above rules. Otherwise, it should be `Error msg` where `msg` describes the error. This time around we won't be checking your messages, but it's good practice to write "reasonable" error messages.

Note that `Ctxt.t` is implemented as a *map*. This is similar to a Python dictionary. The required functions are exposed in the `Ctxt` module. See the type signatures and comments for details.

Semantics

You'll be implementing a big-step CBV semantics. The rules are defined over the same abstract syntax as above. We take the following to be values.

$$\begin{aligned} n &::= \mathsf{Z} \mid \mathsf{S}(n) \\ v &::= \lambda x^\tau.e \mid n \end{aligned}$$

Semantic rules:

$$\frac{}{\lambda x^\tau.e \Downarrow \lambda x^\tau.e}\ \textsc{funE} \qquad \frac{e_1 \Downarrow \lambda x^\tau.e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}\ \textsc{appE}$$

$$\frac{e_1 \Downarrow v_1 \qquad [v_1/x]e_2 \Downarrow v_2}{\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \Downarrow v_2}\ \textsc{letE} \qquad \frac{}{\mathsf{Z} \Downarrow \mathsf{Z}}\ \textsc{zeroE} \qquad \frac{e \Downarrow v}{\mathsf{S}(e) \Downarrow \mathsf{S}(v)}\ \textsc{succE}$$

$$\frac{e_1 \Downarrow \mathsf{Z} \qquad e_2 \Downarrow v_2}{\mathsf{rec}\mathbb{N}(e_1, e_2, e_3) \Downarrow v_2}\ \textsc{recZero}$$

$$\frac{e_1 \Downarrow \mathsf{S}(v_1) \qquad e_3\ v_1\ \mathsf{rec}\mathbb{N}(v_1, e_2, e_3) \Downarrow v}{\mathsf{rec}\mathbb{N}(v_1, e_2, e_3) \Downarrow v}\ \textsc{recSucc}$$

You need to implement:

```
val eval : Expr.t -> Expr.t
```

so that `eval e` is the result of evaluating e according to the above rules, i.e., it should be v such that $e \Downarrow v$ is derivable (since we're only evaluating well-typed expressions, we know v exists and is unique). This depends on the function:

```
val subst : Expr.t -> string -> Expr.t -> Expr.t
```

where `subst e2 x e1` is $[e_2/x]e_1$. Since we're only evaluating closed terms, and we're not evaluating under binders, *this does not have to be capture-avoiding.*

Natural numbers

We've introduced natural numbers (i.e., nonnegative integers) into our language in order to write slightly more interesting programs. As noted above in specification of values, a natural number value is any number of "S" terms stacked on a "Z" term, e.g.,

$$\mathsf{S}(\mathsf{S}(\mathsf{S}(\mathsf{S}(\mathsf{Z}))))$$

represents the number 4 (and our interpreter is designed to display terms in this way).

The natural number recursor "$\mathsf{rec}\mathbb{N}$" is like a "folding" operator for natural numbers. The semantics are define so that

$$\begin{aligned}\mathsf{rec}\mathbb{N}(4, b, f) &= f\ 3\ \mathsf{rec}\mathbb{N}(3, f, b)) \\ &= f\ 3\ (f\ 2\ \mathsf{rec}\mathbb{N}(2, f, b)) \\ &= f\ 3\ (f\ 2\ (f\ 1\ \mathsf{rec}\mathbb{N}(1, f, b))) \\ &= f\ 3\ (f\ 2\ (f\ 1\ (f\ 0\ \mathsf{rec}\mathbb{N}(0, f, b)))) \\ &= f\ 3\ (f\ 2\ (f\ 1\ (f\ 0\ b)))\end{aligned}$$

In other words, evaluating $\mathsf{rec}\mathbb{N}(n, b, f)$ is the sams as applying the function f n times to b (also passing an additional number argument, mostly for convenience). So, for example, if we take b to be 5 and f to be $\lambda p^{\mathbb{N}}.\lambda x^{\mathbb{N}}.\mathsf{S}(x)$ then

$$\begin{aligned}f\ 3\ (f\ 2\ (f\ 1\ (f\ 0\ b))) &= \mathsf{S}(\mathsf{S}(\mathsf{S}(\mathsf{S}(5)))) \\ &= 9\end{aligned}$$

This means we can define an addition function as

$$\lambda m^{\mathbb{N}}.\lambda n^{\mathbb{N}}.\mathsf{rec}\mathbb{N}(m, n, \lambda p^{\mathbb{N}}.\lambda x^{\mathbb{N}}.\mathsf{S}(x))$$

or in our surface syntax:

```
(let add
  (lambda m Nat
    (lambda n Nat
      (let step (lambda p Nat (lambda x Nat (add1 x)))
      (rec-Nat m n step))))
((add 15) 27))
```

which, if you put in file called `example.stlc`, you can run using:

```
# dune exec assign8 example.stlc
Success: 42
```

You need to implement:

```
val multiply : Expr.t
```

that evaluates to a *function* that multiplies two numbers together. I'd recommend first writing the expression in a file called `multiply.stlc` and interpreting it as above. Then writing the expression into `assign8.ml` via the function `Expr.of_string`.

Challenge worth nothing. Also implement the function that takes the *maximum* of two numbers. For a bonkers challenge, implement the function determines the nth number in the Fibonacci sequence.[1]

[1] To be clear, I haven't done this. It's in principle possible, but a waste of time, albeit a fun one.

Written Problems

You're required to submit solutions to the following problems from the lecture notes on the simply-typed λ-calculus (from October 29): **1**, 4, **5**, 7